

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра	<u>Н2</u>	<u>Программная инженерия и интеллектуальные системы</u>
	шифр	наименование кафедры, по которой выполняется работа
Дисциплина	<u>Модели анализа и проектирования программного обеспечения</u>	<u></u>
		наименование дисциплины

УЧЕБНО-ПРАКТИЧЕСКАЯ РАБОТА
№3

номер задания (при наличии)

Модель реализации

при наличии указать тему учебно-практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы О729Б

Барышников О.А.

подпись

фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

Верхова А.А.

подпись

фамилия и инициалы

Оценка / балльная оценка

дата проверки

СОДЕРЖАНИЕ

1	Цель работы и постановка задачи	3
1.1	Цель работы	3
1.2	Постановка задачи и ход выполнения работы	3
2	Выполнение работы	4
2.1	Обоснование выбора диаграммы компонентов реализации.....	4

1 Цель работы и постановка задачи

1.1 Цель работы

Овладеть навыком составления модели реализации с использованием принятых нотаций для построения соответствующих различным подходам диаграмм.

1.2 Постановка задачи и ход выполнения работы

В качестве варианта работы используется тема выпускной квалификационной работы обучающегося.

Подход к проектированию должен быть аналогичен применённому при выполнении предыдущей работы.

Ход выполнения работы состоит из следующих шагов.

Шаг 1. Построить модель реализации ПО, состоящую из диаграммы и описания структуры ПО. Выбор нотации и/или вида диаграммы должен быть обоснован. В случае разработки пользовательского интерфейса допустимо построение диаграммы состояний или графа переходов.

Шаг 2. Построенная диаграмма должна быть дополнена текстовым описанием структуры разработанного ПО/интерфейса (может включать: используемые паттерны, тип архитектуры, структура или тип интерфейса).

Шаг 3. Выбранный тип диаграмм и/или нотации должен быть обоснован. В случае выбора объектно-ориентированного подхода допускается обосновывать исключительно тип диаграммы, а не её нотацию.

Шаг 4. Оформить отчет в соответствии с ГОСТ 7.32-2017. Содержимое отчета – цель работы, тема ВКР, обоснование выбора подхода, построенная модель.

Шаг 5. Загрузить отчет в ЭИОС.

Шаг 6. Дождаться проверки отчета в ЭИОС.

2 Выполнение работы

Тема выпускной квалификационной работы (ВКР) – «Разработка инструмента для автоматической миграции монолитного приложения на микросервисную архитектуру».

Выбранный в ходе выполнения практической работы №1 подход к проектированию – объектно-ориентированный.

В ходе составления модели реализации было решено использовать диаграмму компонентов реализации в нотации UML.

2.1 Обоснование выбора диаграммы компонентов

Выбор диаграммы компонентов реализации обоснован возможностью продемонстрировать конечную физическую структуру исходного кода разработанного программного продукта и отобразить принципы взаимодействия программных модулей (файлов) между собой для дальнейшей поддержки продукта.

Диаграмма компонентов представляет из себя совокупность файлов исходного кода, предоставляемых интерфейсов (классов), отношений ассоциации и зависимости. Компонентом считается некоторая отдельная часть системы с определённой функцией и ролью. Интерфейс определяет порядок взаимодействия с компонентом. С помощью отношений ассоциации и зависимости указываются связи между компонентами.

На рисунке 1 представлена диаграмма компонентов реализации. На диаграмме видно, что система разделена на несколько обособленных компонентов, каждый из которых предоставляет для работы с собой интерфейс в виде класса.

Ядро системы и графический интерфейс располагаются в файле main.py. В нём происходит инициализация системы, отрисовка окон, обработка пользовательских событий (нажатия кнопок) и обращение к остальным элементам (модулям) системы.

В файле `scanner.py` располагается логика лексического и синтаксического анализа (построение AST-дерева) и эвристического поиска границ микросервисов. Предоставляемый интерфейс – класс `Scanner`.

В файле `generator.py` располагается логика физического разделения кода, очистки моделей баз данных от связей `ForeignKey` и рендеринга инфраструктурных конфигураций. Предоставляемый интерфейс – класс `Generator`.

В файле `docker_manager.py` располагается логика по асинхронному взаимодействию с системной службой `Docker Engine` посредством вызовов `subprocess`. Предоставляемый интерфейс – класс `DockerManager`.

Также на диаграмме представлены зависимости от внешних библиотек, интерфейсы которых импортируются в программный код: библиотека `customtkinter` для построения современного GUI, библиотека `jinja2` для работы с шаблонами файлов и стандартный модуль `ast` для разбора синтаксиса Python. Результатом работы системы являются генерируемые артефакты — файлы конфигураций `docker-compose.yaml` и `Dockerfile`.

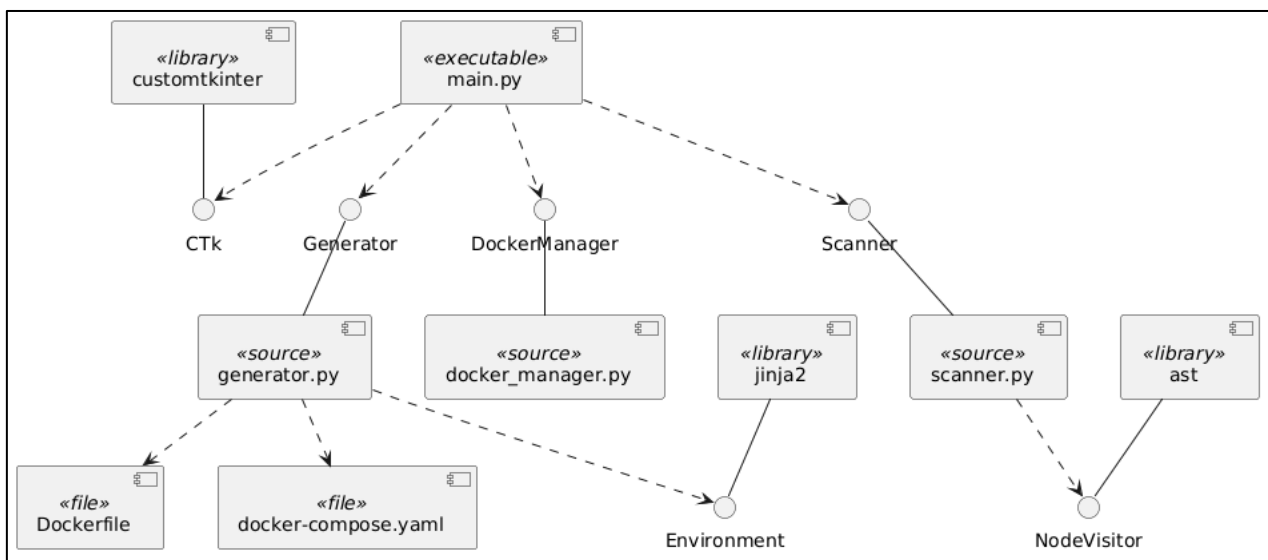


Рисунок 1 – Диаграмма компонентов